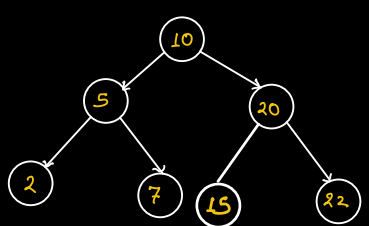
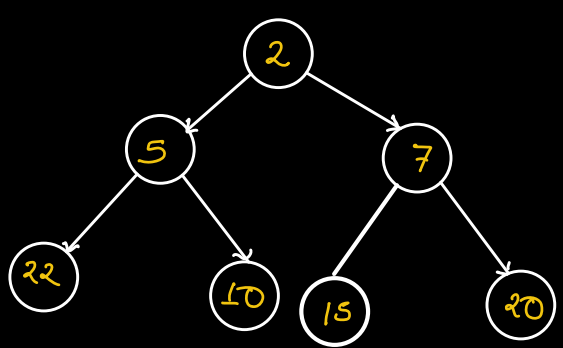
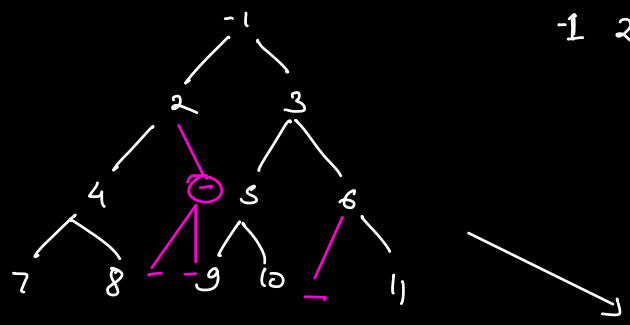
	insert(x)	getMin()	deleteMin()
Sorted Array	O(N)	O(1)	O(N) ASC O(1) DESC
Linked list (Sorted)	O(N)	O(1)	O(1)
Binary Search Tree	O(N)	O(N)	O(N)
Balanced Binary Search tree	O(logN)	O(logN)	O(logN)
Min-Heap	O(logN)	O(1)	O(logN)



BBST

2 5 7 10 15 20 22

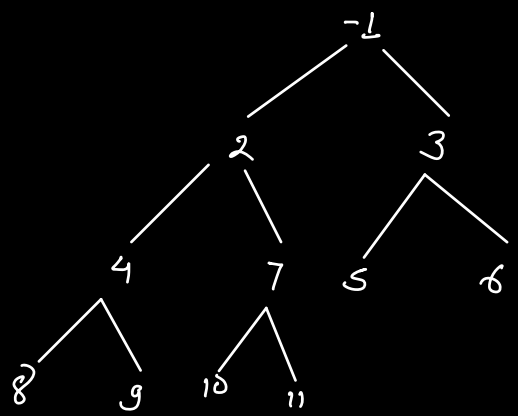




-1 2 3 4 _ 5 6 7 8 _ _ 9 10 _ 11

Complete Binary Tree

Ht → log N

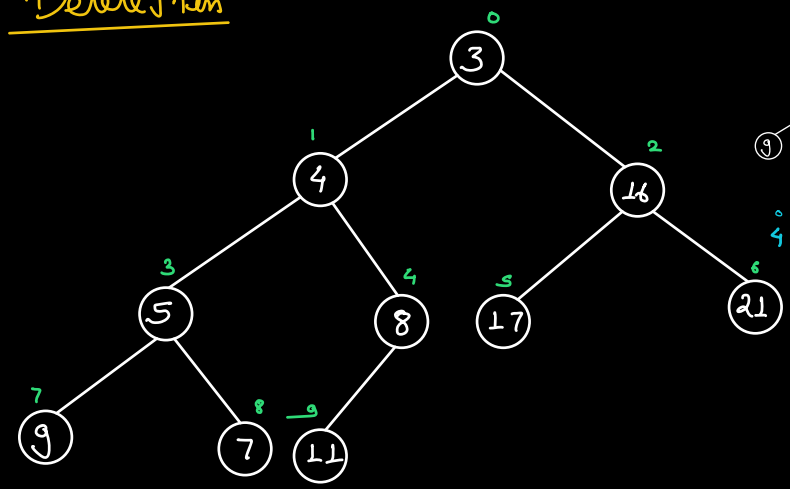


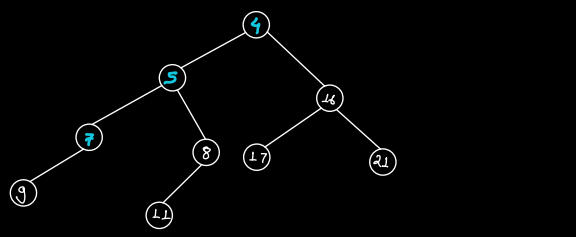
-1 2 3 4 7 5 6 8 9 10 11

Heap

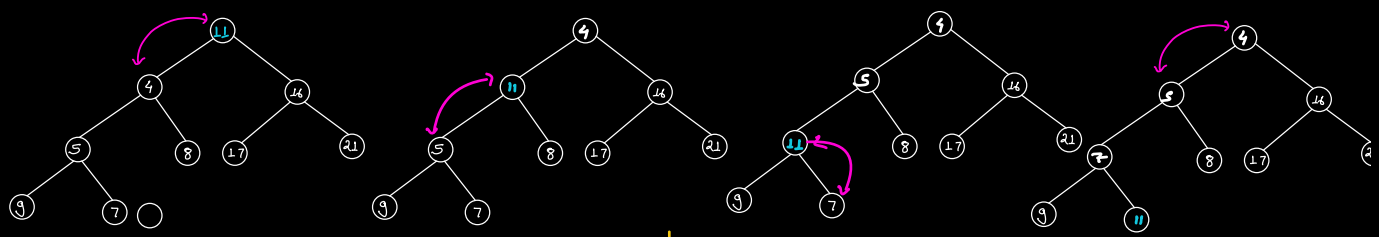
- Complete BT
- Min / Max property

Delete Min

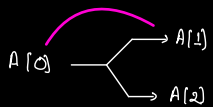


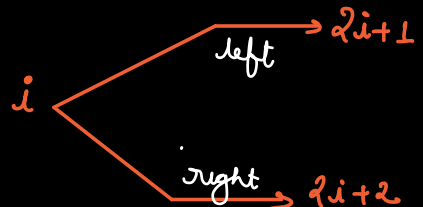


0	1	2	3	4	5	6	7	8	9
3	4	16	5	8	17	21	9	7	11

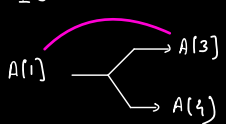


0	1	2	3	4	5	6	7	8	9
11	4	16	5	8	17	21	9	7	3

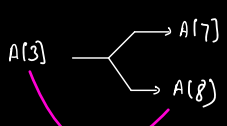




0	1	2	3	4	5	6	7	8	9
4	11	16	5	8	17	21	9	7	3



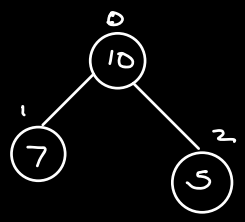
0	1	2	3	4	5	6	7	8	9
4	5	16	11	8	17	21	9	7	3



0	1	2	3	4	5	6	7	8	9
4	5	16	7	8	17	21	9	11	3

Delete the min from a given min heap (Array)

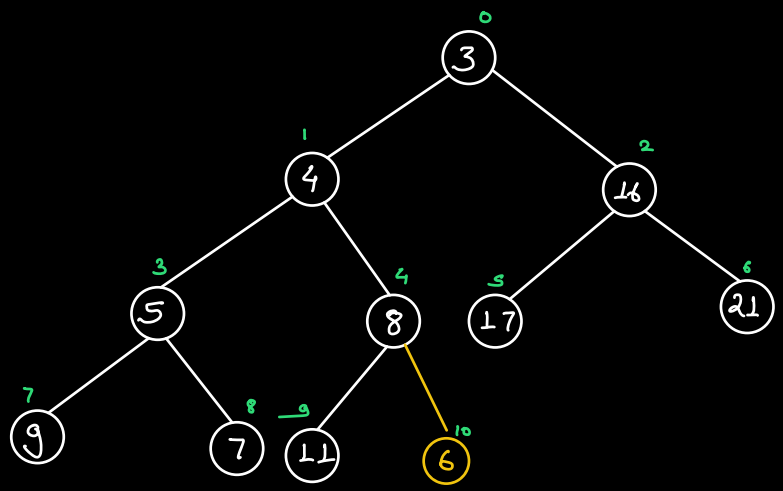
```
int deleteMin( A ) {
    ans = A[0];
    swap( A[0], A[last] );
    last--;
    i = 0;
    while ( i < last ) {
        int minIdx = i;
        int l = 2i+1 , r = 2i+2;
        if ( l <= last && A[l] < A[minIdx] ) {
            minIdx = l;
        }
        if ( r <= last && A[r] < A[minIdx] ) {
            minIdx = r;
        }
        if ( minIdx == i ) {
            break;
        }
        swap( A[i], A[minIdx] );
        i = minIdx;
    }
    ret ans;
}
```



Percolate down
or
Shift down

O(log N)

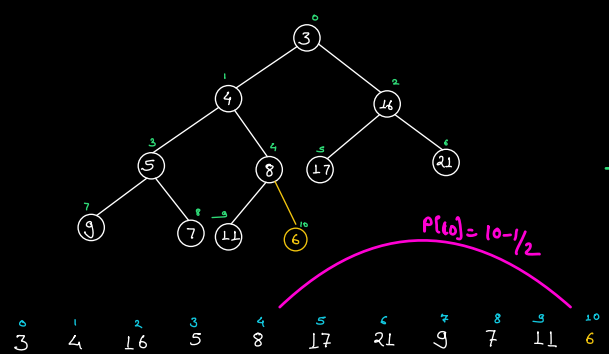
Insert in heap

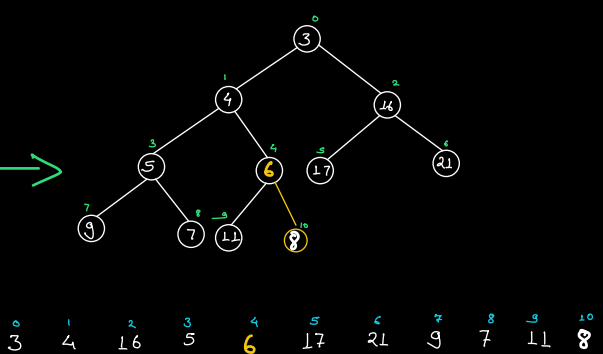


Percolate up
or
Shift up

last

0	1	2	3	4	5	6	7	8	9	10
3	4	16	5	8	17	21	9	7	11	6





Parent(i) ⟶ (i-1)/2

~~O(N log N)~~

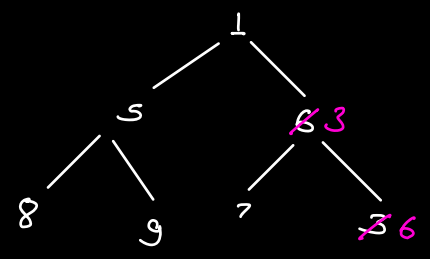
O(N)

HW

1, 5, 6, 8, 9, 7, 3

↓

1, 5, 3, 8, 9, 7, 6



TC : O(N)

Priority Queue (Min Heap) ⟶ Heap (delete Min) (get Min)

add (insert), poll (deleteMin), peek (getMin)

```
PriorityQueue<Int> heap = new PriorityQueue<Int>();
```

Q Given an array. Find the K smallest elements.

7, 9, 2, 1, 10, 3, 5, 8, 12, 50

K=5 7, 2, 1, 3, 5

Approach 1

- Sort the array
- Return first K elements.

TC: O(N logN)

Approach 2

- Array ⟶ Min heap — O(N)
- deleteMin() ⟶ K times — O(K logN)

TC: O(N) + O(K logN)

SC: O(N)

7, 9, 2, 1, 10, 3, 5, 8, 12, 50

K=5 7, 2, 1, 3, 5

7 2 1 3 5 9, 10, 8, 12, 50

↓

7, 9, 2, 1, 10, 3, 5, 8, 12, 50

7 9 2 1 10 ⟶ 10 > 3
7 9 2 1 3 ⟶ 9 > 5
7 5 2 1 3 ⟶ 7 < 8

Max heap of size K

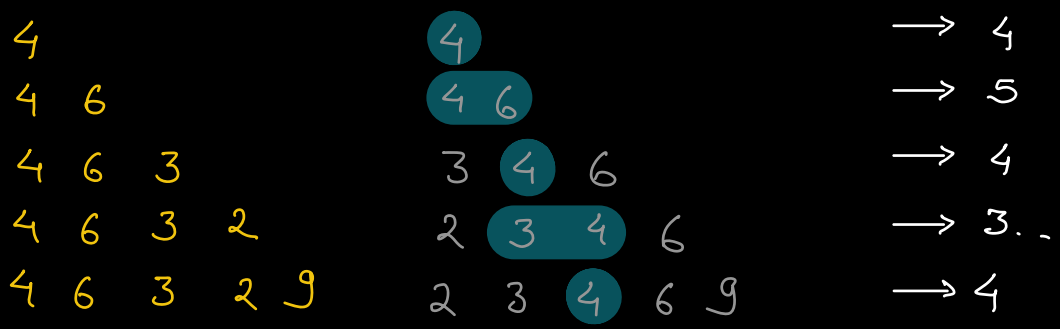
- Build a max heap with first K elements. O(K)
- For every i in [K+1 to N-1] N-K
  if (A[i] < heap.getMax()) {
    → deleteMax() } O(log K)
    → insert(A[i]); } (N-K) log K

TC: O(K) + O(N-K log K)
SC: O(K)

Amazon
MS
Google
Facebook
⋮

Running Median

Getting a stream of integers.
With every new element return the current median.



Approach 1

Maintain data in sorted order
(Insertion sort)

TC / insertion → O(N)

↓ optimize

EM / IC